Chapter 28 (Locks) — Homework (Simulation)

Zhiyuan Pan

All runs use `x86.py` in `ostep-homework/threads-locks/`. This version adds `-P` (a manual schedule: each character is the thread that runs the next instruction, and the string repeats) and the `%ex` / `%fx` registers, `xchg`, `fetchadd` and `yield`. Every experiment below can be rerun with `code/chapter28/run.sh`.

Long runs (`bx=100` or `bx=1000`) produce thousands of trace lines, so I summarize them with `code/chapter28/lock_stats.py`. It runs the official simulator with `-c` and only reads the trace. For each run it reports:

field	meaning
`count`	final value of the shared counter
`cs`	how many times some thread entered the critical section; **`count == cs` means no update was lost**
`max_in_cs`	the most threads inside the critical section (between its first instruction and the release) at the same time; **more than 1 means mutual exclusion is broken**
`instr`	instructions executed in total
`spin`	instructions spent in the acquire code beyond the uncontended fast path, i.e. **wasted waiting for the lock**
`xchg`, `yield`	atomic exchanges / yields executed
`maxwait`	longest time (instructions, all threads) any thread waited from starting to acquire until it got the lock

The `lock_stats.py schedules` mode runs hundreds of random `-P` schedules (seeded, so they are reproducible) and counts how many break mutual exclusion, lose an update, or never finish.

Q1

> Examine `flag.s`. This code "implements" locking with a single memory flag. Can you understand the assembly?

```
.acquire
mov  flag, %ax      # read the flag
test $0, %ax        # is it 0 (free)?
jne  .acquire       # no: keep reading (spin)
mov  $1, flag       # yes: set it to 1 (taken)
# critical section
mov  count, %ax
add  $1, %ax
mov  %ax, count
# release lock
mov  $0, flag
# loop bx times
sub  $1, %bx
test $0, %bx
jgt .top
halt
```

`flag` is 0 when the lock is free and 1 when it is held. To acquire, a thread spins until it reads 0 and then writes 1. To release, it writes 0. The critical section increments the shared `count`, and the whole thing repeats `%bx` times. One iteration is **11 instructions**.

The problem: **testing the flag and setting it are two separate instructions**. If a thread is interrupted after it has read 0 (`mov flag,%ax`) but before it has written 1 (`mov $1,flag`), the other thread can also read 0 and also enter. This is the broken first attempt from Section 28.6 of the chapter.

Q2

> When you run with the defaults, does `flag.s` work? Use the `-M` and `-R` flags to trace variables and registers (and turn on `-c` to see their values). Can you predict what value will end up in `flag`?

Yes, with the defaults it works. It runs two threads with `bx = 0` (one iteration each) and the default interval of 50. One iteration is only 12 instructions including `halt`, so thread 0 runs completely before thread 1 starts. The flag is never contended:

```
 flag count      ax    bx         Thread 0                Thread 1
    0     0       0     0   1000 mov  flag, %ax
    0     0       0     0   1001 test $0, %ax
    0     0       0     0   1002 jne  .acquire
    1     0       0     0   1003 mov  $1, flag
    1     1       1     0   1006 mov  %ax, count
    0     1       1     0   1007 mov  $0, flag
    0     1       1    -1   1011 halt
    0     1       0     0   ----- Halt;Switch -----  ----- Halt;Switch -----
    1     1       0     0                           1003 mov  $1, flag
    1     2       2     0                           1006 mov  %ax, count
    0     2       2     0                           1007 mov  $0, flag
    0     2       2    -1                           1011 halt
```

`count` ends at **2**, and **`flag` ends at 0** (each thread sets it to 1 and clears it again before halting). In general `flag` always ends at 0, because the last thing every thread does with it is the release.

Q3

> Change the value of the register `%bx` with the `-a` flag (e.g., `-a bx=2,bx=2` if you are running just two threads). What does the code do? How does it change your answer for the question above?

`%bx` is the **loop count**: each thread now acquires the lock, increments `count` and releases it **twice**. That is 2 × 11 + 1 = 23 instructions per thread, still fewer than 50, so the threads still run one after the other:

```
count=4  cs=4  max_in_cs=1  instr=46  spin=0
```

The answer barely changes: `count` ends at **4** (= 2 threads × 2 iterations) instead of 2, and `flag` still ends at **0**. The lock still looks correct, but only because nothing interleaves.

Q4

> Set `bx` to a high value for each thread, and then use the `-i` flag to generate different interrupt frequencies; what values lead to a bad outcomes? Which lead to good outcomes?

With `-a bx=100,bx=100` the correct result is `count = 200`. Sweeping `-i 1` to `-i 40`:

`-i`	count	max in CS		`-i`	count	max in CS
1	100	2		11	**200**	**1**
2	100	2		12	176	2
3	133	2		15	**200**	**1**
4	167	2		16	178	2
5	175	2		21	**200**	**1**
6	183	2		22	**200**	**1**
7	166	2		31	200	2
8	160	2		32	200	2
9	188	2		33	**200**	**1**
10	193	2		40	176	2

(Every other interval from 1 to 40 also gives a wrong count. All 40 rows are in the `run.sh` output.)

- **Good outcomes: `-i 11, 15, 21, 22, 33`.** Mutual exclusion holds and no update is lost. 11 is the length of one iteration, and 22 and 33 are multiples of it. A thread is then always interrupted at the end of an iteration, never between reading and setting the flag. 15 and 21 happen to line up in a way that also never splits the test from the set.
- **Bad outcomes: everything else.** Both threads get past the flag check together (`max_in_cs = 2`), and usually updates are lost. `-i 1` and `-i 2` lose half of them (count = 100): the threads run in lock step, so they read the flag together and also read `count` together.
- **`-i 31` and `-i 32` are a trap.** `count` is the correct 200, but both threads were in the critical section at the same time. They just did not interleave on the `count` load and store. A correct final value does not prove the lock works.

As in chapter 26, the outcome depends on how the interval lines up with the code, not on how large it is. `flag.s` is **not a lock**.

Q5

> Now let's look at the program `test-and-set.s`. First, try to understand the code, which uses the `xchg` instruction to build a simple locking primitive. How is the lock acquire written? How about lock release?

```
.acquire
mov  $1, %ax
xchg %ax, mutex     # atomically: old = mutex; mutex = 1; ax = old
test $0, %ax        # was the old value 0?
jne  .acquire       # no: someone holds it, try again
# critical section
mov  count, %ax
add  $1, %ax
mov  %ax, count
# release lock
mov  $0, mutex
```

Acquire: put 1 in `%ax` and **atomically swap** it with `mutex`. After the `xchg`, `mutex` is 1 no matter what, and `%ax` holds the old value. If the old value was 0, the lock was free and this thread now holds it. If it was 1, someone else holds it (and writing 1 again changed nothing), so the thread loops and tries again: a **spin lock**. Because the read and the write happen in **one** atomic instruction, no interrupt can come between them. That is exactly the gap that broke `flag.s`.

Release: a single ordinary store `mov $0, mutex`. Only the holder releases, and a single store is atomic, so nothing more is needed.

Q6

> Now run the code, changing the value of the interrupt interval (`-i`) again, and making sure to loop for a number of times. Does the code always work as expected? Does it sometimes lead to an inefficient use of the CPU? How could you quantify that?

Same sweep as Q4, with `-a bx=100,bx=100`:

`-i`	count	max in CS	total instr	spin instr	spin share
1	200	1	3002	800	27%
5	200	1	2998	796	27%
11	200	1	2202	**0**	0%
12	200	1	3858	1656	43%
22	200	1	2202	**0**	0%
34	200	1	3870	1668	43%
50	200	1	3354	1152	34%
100	200	1	3594	1392	39%

It always works. For every interval from 1 to 40 the count is 200 and there is never more than one thread in the critical section.

But it often wastes the CPU. When a thread is interrupted while it holds the lock, the other thread runs and **spins for its entire time slice** (mov/xchg/test/jne over and over). It cannot get the lock, because the holder is not running and so cannot release it.

How to quantify it: count the instructions executed in the acquire loop beyond the 4 that an uncontended acquire needs (the `spin` column), and divide by the total. With 2 threads that is 27% to 43% of all instructions for most intervals. It is 0 only for -i 11, 22, 33, where slices end exactly at iteration boundaries, so the lock is never held across a switch. Other ways to measure the same thing: the number of failed `xchg` (here 200 succeed, the rest fail; at -i 50 there are 488 `xchg` in total), or the number of time slices that end without any progress.

Q7

> Use the `-P` flag to generate specific tests of the locking code. For example, run a schedule that grabs the lock in the first thread, but then tries to acquire it in the second. Does the right thing happen? What else should you test?

The suggested test: `-P 0011111111000000000 -a bx=1,bx=1`. Thread 0 runs `mov` and `xchg` (it now holds the lock), then thread 1 runs 8 instructions:

```
mutex count      ax          Thread 0                Thread 1
    0     0       1   1000 mov  $1, %ax
    1     0       0   1001 xchg %ax, mutex                      (a)
    1     0       1                    1000 mov  $1, %ax
    1     0       1                    1001 xchg %ax, mutex     (b)
    1     0       1                    1002 test $0, %ax
    1     0       1                    1003 jne  .acquire
    1     0       1                    1000 mov  $1, %ax
    1     0       1                    1001 xchg %ax, mutex     (b)
    1     0       1                    1002 test $0, %ax
    1     0       1                    1003 jne  .acquire
    1     0       0   1002 test $0, %ax
    1     0       0   1003 jne  .acquire
    1     1       1   1006 mov  %ax, count
    0     1       1   1007 mov  $0, mutex                      (c)
    ...
    1     1       0                    1001 xchg %ax, mutex     (d)
    1     2       2                    1006 mov  %ax, count
```

- (a) T0's `xchg` returns 0: T0 holds the lock.

- (b) T1's `xchg` returns 1 (`%ax = 1`), so T1 spins.
- (c) T0 releases the lock.
- (d) T1's next `xchg` returns 0 and it enters.

Yes, the right thing happens. Thread 1's `xchg` returns 1 as long as thread 0 holds the lock, so it keeps spinning and never enters. It gets the lock only after thread 0's release, and the count ends at 2.

Other things worth testing:

test	schedule	result
both threads race for the `xchg` (alternate every instruction)	`-P 01`	count = 2, max in CS = 1; T0's `xchg` comes first and wins, T1 spins until the release
holder is preempted **inside** the critical section	`-P 000001111111111111`	count = 2, max in CS = 1; T1 spins 12 extra instructions
holder releases, then the other thread acquires	(in both runs above)	works: the waiter's next `xchg` returns 0
300 random schedules, 2 threads, `bx=3`	`lock_stats.py schedules`	**0** broke mutual exclusion, **0** lost updates, **0** hung
300 random schedules, **3** threads, `bx=2`		**0 / 0 / 0**
the same 300 schedules on **`flag.s`** (to check that the test can catch a broken lock)		**208** broke mutual exclusion, **171** lost updates (first bad one: `-P 001010011011`)

The last row matters: the random-schedule test does find bugs, so its "0" for test-and-set means something. Still, random testing is not a proof. A full check would also cover more threads, a thread trying to release a lock it does not hold, and fairness (which test-and-set does not guarantee, see Q12).

Q8

> Now let's look at the code in `peterson.s`, which implements Peterson's algorithm (mentioned in a sidebar in the text). Study the code and see if you can make sense of it.

```
lea flag, %fx           # fx = &flag[0]
mov %bx, %cx            # bx = self (0 or 1)
neg %cx
add $1, %cx             # cx = 1 - self = other
.acquire
mov $1, 0(%fx,%bx,4)    # flag[self] = 1      "I want to enter"
mov %cx, turn           # turn = other        "but you go first"
.spin1
mov 0(%fx,%cx,4), %ax   # if flag[other] != 1: enter
test $1, %ax
jne .fini
.spin2
mov turn, %ax           # if turn != other: enter
test %cx, %ax
je .spin1               # else (other wants in AND it is other's turn): spin
.fini
mov count, %ax          # critical section
add $1, %ax
mov %ax, count
.release
mov $0, 0(%fx,%bx,4)    # flag[self] = 0
mov %cx, turn
halt
```

This is the C code from the chapter's sidebar:

```
flag[self] = 1; turn = other;
while (flag[other] == 1 && turn == other) ;   // spin
/* critical section */
flag[self] = 0;
```

It uses **only ordinary loads and stores**, no atomic instruction. Each thread has a `flag` that says "I want to enter". The shared `turn` breaks ties: each thread politely gives the turn to the other, and **whoever writes `turn` last waits**. A thread enters if the other does not want to, or if it is not the other's turn. There is no loop over `bx` here, since `bx` is the thread ID, so each thread enters the critical section once and the expected count is 2.

Q9

> Now run the code with different values of `-i`. What kinds of different behavior do you see? Make sure to set the thread IDs appropriately (using `-a bx=0,bx=1` for example) as the code assumes it.

With `-a bx=0,bx=1` and `-i 1` to `-i 20`, **every run is correct**: count = 2 and at most one thread in the critical section. What changes is how much the threads wait:

`-i`	total instr	spin instr	what happens
1, 2	45	15	both threads announce interest almost together; the one that wrote `turn` last spins until the other finishes
3	42	12	similar
4	36	6	
5	**54**	**24**	the most waiting: a thread is interrupted while the other is still in the critical section and has to spin for whole slices
6	39	9	
7–9	33	3	the second thread checks once or twice and then enters
10–12	36	6	
≥ 13	30	**0**	each thread runs 15 instructions; within its first 13 thread 0 gets through acquire, the critical section and clears `flag[0]`, so thread 1 never has to wait

So the observable behaviors are: no contention at all (large `-i`), short waits where one thread spins a few times, and long waits when the thread inside the critical section is descheduled. It never breaks mutual exclusion and never deadlocks.

With wrong IDs (e.g. the default `bx=0` for both threads) the code is not valid, because both threads use `flag[0]`.

Q10

> Can you control the scheduling (with the `-P` flag) to "prove" that the code works? What are the different cases you should show hold? Think about mutual exclusion and deadlock avoidance.

The cases to show:

1. No contention. `-P 00000000000000000011`: thread 0 runs its whole acquire and critical section before thread 1 does anything. `flag[1] = 0`, so thread 0 enters immediately (spin = 0), and thread 1 later does the same. A lock must not make a thread wait when nobody else wants it.

2. Both threads want in at the same time: mutual exclusion and no deadlock. `-P 01` alternates every instruction, so both set their flag and both write `turn` before either one checks:

```
flag  turn count      cx          Thread 0                Thread 1
   1     0     0       1   1004 mov $1, 0(%fx,%bx,4)                   (a)
   1     0     0       0                    1004 mov $1, 0(%fx,%bx,4) (a)
   1     1     0       1   1005 mov %cx, turn                         (b)
   1     0     0       0                    1005 mov %cx, turn        (b)
   ...
   1     0     0       1   1011 je .spin1                             (c)
   1     0     0       1   1012 mov count, %ax
   1     0     0       0                    1006 mov 0(%fx,%cx,4),%ax (d)
   1     0     1       1   1014 mov %ax, count
   0     0     1       1   1015 mov $0, 0(%fx,%bx,4)                  (e)
   ...
   0     1     1       0                    1008 jne .fini            (f)
   0     1     2       0                    1014 mov %ax, count
```

(The `flag` column shows `flag[0]`.)

- (a) Both threads set their flag: `flag[0] = flag[1] = 1`.
- (b) T0 writes `turn = 1`, then T1 writes `turn = 0`. T1 is the last writer.
- (c) Both see `flag[other] = 1` and check `turn`. For T0, `turn` (0) is not `other` (1), so T0 falls through into the critical section.
- (d) For T1, `turn == other`, so it goes back to `.spin1` and spins.
- (e) T0 releases: `flag[0] = 0`.
- (f) T1 now sees `flag[0] = 0` and enters.

Thread 1 wrote `turn` last, so thread 1 waits and thread 0 enters. Only one of them can get in. This is the **mutual-exclusion** case. Neither waits forever, because `turn` has only one value: it cannot be "the other's turn" for both threads at once, so at least one of them always gets through. This is the **deadlock-avoidance** case.

3. One thread arrives while the other is in the critical section. `-P 0000000000111111111`: thread 0 is inside when thread 1 starts. Thread 1 sees `flag[0] = 1` and `turn = 0`, so it spins (3 instructions) until thread 0's release clears `flag[0]`, then enters. count = 2, max in CS = 1.

4. The waiting thread really gets in after the release (bounded waiting). Shown in cases 2 and 3: once the holder clears its flag the waiter's next check succeeds. The holder cannot jump back ahead either, because to re-enter it would set `turn = other` and then wait itself.

5. Many interleavings. 500 random 12-slot schedules and 500 random 6-slot schedules: **0** broke mutual exclusion, **0** lost an update, **0** failed to finish.

These tests cover every case of the standard proof (the other thread is not interested / both are interested and `turn` decides / one is already inside). But a set of schedules only **tests** the code; it does not prove it. The real proof is the argument in case 2: if both threads were inside, both would have seen `turn` pointing to themselves, which is impossible. And the simulator executes every load

and store in order (sequential consistency). On real multiprocessors with store buffers, Peterson's algorithm needs memory fences to work.

Q11

> Now study the code for the ticket lock in `ticket.s`. Does it match the code in the chapter? Then run with the following flags: `-a bx=1000,bx=1000` (causing each thread to loop through the critical section 1000 times). Watch what happens; do the threads spend much time spin-waiting for the lock?

```
.acquire
mov $1, %ax
fetchadd %ax, ticket  # myturn = FetchAndAdd(&ticket, 1)   (old value -> ax)
.tryagain
mov turn, %cx         # while (turn != myturn) spin
test %cx, %ax
jne .tryagain
# critical section ...
mov $1, %ax
fetchadd %ax, turn    # unlock: FetchAndAdd(&turn, 1)
```

Yes, it matches Figure 28.7 of the chapter: `lock()` takes a ticket with an atomic fetch-and-add and spins until `turn` equals that ticket; `unlock()` increments `turn`. (The chapter's unlock is `lock->turn = lock->turn + 1`; here it is also a `fetchadd`, which makes no difference since only the holder writes `turn`.)

With `-a bx=1000,bx=1000` (default `-i 50`) the result is correct, but **most of the time is spent spinning**:

```
count=2000  cs=2000  max_in_cs=1  instr=99463  acquire=83461  spin=73461  maxwait=103
```

74% of all instructions are spin-waiting. The reason is visible in the trace: a thread finishes one critical section, loops, and takes a **new** ticket. That ticket is after the other thread's, and the other thread is not running. So the thread has to spin for the rest of its time slice. Each time slice therefore performs **only one critical section** (about 50 instructions per increment, versus 11 without contention). The strict FIFO order forces the threads to alternate, even though only one of them runs at a time.

The longer the time slice, the worse it gets:

`-i`	total instr	spin instr	spin share
10	33,328	7,326	22%
50	99,463	73,461	74%
100	198,718	172,716	87%
500	962,521	936,519	97%

Q12

> How does the code behave as you add more threads?

`bx=100` per thread, default `-i 50`, compared with test-and-set:

threads	ticket: instr	instr per increment	maxwait	test-and-set: instr	instr per increment	maxwait
2	9,463	47	103	3,354	17	204
3	14,424	48	153	7,287	24	704
4	19,286	48	203	11,856	30	1,204
6	29,010	48	303	25,870	43	3,004
8	38,836	49	**403**	45,016	56	**5,604**

(All runs: count = threads × 100, max in CS = 1.)

- **The ticket lock is always correct and fair.** Threads get the lock in ticket order, so a waiting thread is passed over by at most one critical section of each other thread. Its longest wait grows only linearly, about (threads − 1) × 50 instructions (one time slice per other thread): 103, 153, 203, 303, 403.
- **Its cost per increment stays flat at about 48 instructions** (one time slice of spinning per critical section, as in Q11). The total work grows linearly with the number of threads.
- **Test-and-set is cheaper with few threads but gets worse quickly.** Its cost per increment rises from 17 to 56 instructions, and with 8 threads it is more expensive than the ticket lock. It is also **unfair**: the longest wait grows from 204 to 704, 1,204, 3,004 and 5,604 instructions, because whichever thread happens to run when the lock is free gets it, and some threads lose many times in a row.

So adding threads does not break the ticket lock, but on one CPU every extra thread is one more time slice that a waiter may spin through before its turn.

Q13

> Now examine `yield.s`, in which a yield instruction enables one thread to yield control of the CPU (realistically, this would be an OS primitive, but for the simplicity, we assume an instruction does the task). Find a scenario where `test-and-set.s` wastes cycles spinning, but `yield.s` does not. How many instructions are saved? In what scenarios do these savings arise?

`yield.s` is test-and-set, except that when the `xchg` fails the thread executes `yield` (switch to another thread right away) instead of trying again:

```
mov  $1, %ax
xchg %ax, mutex
test $0, %ax
je .acquire_done
yield               # lock is busy: give up the CPU
j .acquire
```

Scenario: `-a bx=2,bx=2 -i 16`. Thread 0 runs one full iteration (11 instructions) and starts a second. Its 16th instruction is the load inside the critical section, so it is **interrupted while holding the lock**. Thread 1 then tries to acquire:

```
mutex count          Thread 0                Thread 1
    1     1   1006 mov  count, %ax                      (a)
    1     1   ------ Interrupt ------  ------ Interrupt ------
    1     1                          1000 mov  $1, %ax
    1     1                          1001 xchg %ax, mutex (b)
    1     1                          1002 test $0, %ax
    1     1                          1003 je .acquire_done
    1     1                          1004 yield          (c)
    1     1   ------ Interrupt ------  ------ Interrupt ------
    1     1   1007 add  $1, %ax                       (d)
```

- (a) T0 holds the lock and is interrupted here.
- (b) The lock is busy.
- (c) T1 gives the CPU back at once.
- (d) T0 finishes the critical section and releases the lock.

With `test-and-set.s`, thread 1 would instead spin for its **whole** 16-slot time slice. The same scenario with both programs:

program	total instr	spin instr
`test-and-set.s`	62	16
`yield.s`	52	6

10 instructions are saved: thread 1 uses 5 instructions (plus one `j` when it comes back) instead of 16. In general, each time a waiter finds the lock held by a **descheduled** thread, yield saves roughly *(time slice − 5)* instructions. With longer slices and more threads the savings are large (`bx=100`):

threads, `-i`	test-and-set instr	yield instr	saved
2, 5	2,998	3,006	−8 (none)
2, 16	3,194	2,598	19%
2, 50	3,354	2,334	30%
2, 100	3,594	2,274	37%
4, 50	11,856	5,196	56%
4, 100	13,084	4,656	**64%**

The savings arise when the lock holder has been preempted while other threads want the lock, and they grow with the length of the time slice and the number of waiting threads. With a very short slice (`-i 5`) there is nothing to save: a spinner would be interrupted soon anyway, and the yield costs about as much as the spinning. Yield does not help when the holder is running on another CPU and will release soon. Then spinning briefly is cheaper than a context switch. A real `yield()` is a system call and a context switch, which costs far more than one instruction. With many threads it is also not enough, because a yielding thread can still be scheduled again and again before the holder (the chapter's argument for queues and `park()`).

Q14

> Finally, examine `test-and-test-and-set.s`. What does this lock do? What kind of savings does it introduce as compared to `test-and-set.s`?

```
.acquire
mov  mutex, %ax     # first just READ the lock
test $0, %ax
jne .acquire        # spin with ordinary loads while it is taken
mov  $1, %ax
xchg %ax, mutex     # it looked free: now try the atomic exchange
test $0, %ax
jne .acquire        # someone was faster: back to reading
```

It **spins on an ordinary load** and only attempts the expensive atomic `xchg` when the lock **looks** free. The `xchg` can still fail if another thread takes the lock between the read and the exchange, so the second test is still needed for correctness. Like test-and-set it is always correct (count = threads × 100, max in CS = 1 in every run).

The saving is in **atomic operations**, not instructions (`bx=100`):

threads, `-i`	TAS `xchg`	TTAS `xchg`	TAS instr	TTAS instr
2, 16	448	**220**	3,194	3,842
2, 50	488	**227**	3,354	4,257
2, 100	548	**203**	3,594	3,714
4, 16	1,964	**486**	10,660	10,025
4, 50	2,263	**429**	11,856	14,219
4, 100	2,570	**412**	13,084	9,252

TTAS executes only slightly more `xchg` than the minimum (one per acquisition: 200 or 400). Test-and-set executes 2 to 6 times as many. The instruction count is about the same, sometimes higher, because the TTAS acquire path is longer (7 instructions instead of 4), and in this simulator a load costs the same as an `xchg` .

On a real multiprocessor the difference is large. Every `xchg` is a **write**: it needs the cache line in exclusive state, so each failed `xchg` invalidates the line in all other cores' caches. Spinning test-and-set threads keep bouncing the line between cores and slow down everyone, including the lock holder. TTAS waiters spin on a **read** of their own cached copy (no bus or coherence traffic) until the holder's release invalidates it. So the saving is in **cache-coherence traffic and memory-bus bandwidth**, which a single-CPU instruction simulator cannot show directly. The `xchg` counts above are a proxy for it.